

# From Transformers to Modern LLMs: Pretraining, BERT, GPT, LLaMA, and Efficiency Techniques

## Study Notes

## 1 From Seq2Seq Transformers to BERT (Lecture 4, pp. 54–60)

### 1.1 Why Training a Transformer From Scratch Is Hard (p. 54)

Training a full transformer well requires two things most people do not have: a huge amount of text data, and a huge amount of compute (many GPUs, running for a long time). Training a large model from nothing, every time you have a new task, is not realistic for most practitioners. This motivates the idea introduced on the next slide: someone else does the expensive training once, and everyone else reuses the result.

### 1.2 Pretraining (p. 55)

The core idea: a transformer can first be trained on a generic, self-supervised objective over huge amounts of raw text (no manual labels needed) to learn a general-purpose representation of language. This step is called *pretraining*. Anyone else can then take that pretrained model and adapt it to their own, much smaller, labeled task, instead of training a model from zero. This is exactly the same logic as transfer learning in computer vision (train once on ImageNet, reuse the features everywhere), applied to language.

### 1.3 BERT: Overview and Scale (p. 56)

**Core idea.** BERT (Bidirectional Encoder Representations from Transformers) is built from the *encoder* stack of the original transformer only (no decoder). Its defining property is that every layer conditions on both the left and right context of a token at the same time – it is bidirectional. This is pretrained once on unlabeled text, and then adapted to a target task by adding just one small output layer and fine-tuning.

**Mechanism and sizes.** Denote the number of stacked encoder blocks as  $L$ , the hidden size (width of every token's vector) as  $H$ , and the number of attention heads per block as  $A$ .

- BERT<sub>BASE</sub>:  $L = 12$ ,  $H = 768$ ,  $A = 12$ , total parameters  $\approx 110\text{M}$ .
- BERT<sub>LARGE</sub>:  $L = 24$ ,  $H = 1024$ ,  $A = 16$ , total parameters  $\approx 340\text{M}$ .

These exact numbers are confirmed in the original paper (Devlin et al., *BERT*, Section 3), which also specifies a detail the slide omits: the feed-forward (filter) size inside each block is always  $4H$  –

so 3072 for BASE and 4096 for LARGE. Per-head dimension is  $H/A$ : for BASE,  $768/12 = 64$ ; for LARGE,  $1024/16 = 64$  – both use the same 64-dimensional attention heads, just more of them.

Tokenization uses WordPiece (subword units). The BERT paper (Section on Input/Output Representations) specifies a vocabulary of 30,000 WordPiece tokens. This means the input embedding matrix alone has shape  $[30\,000, H]$ : for BASE that is  $30\,000 \times 768 \approx 23.0\text{M}$  parameters, already about a fifth of the full 110M-parameter budget, before any encoder block is counted.

**Contrast (do not confuse these).** BERT’s encoder self-attention is *bidirectional*: every token can attend to every other token in the sequence, left and right. This is different from the *causal* / *masked* self-attention used by decoder-only models like GPT (covered later), where a token can only attend to tokens at or before its own position. BERT’s paper states this directly: BERT<sub>BASE</sub> was deliberately sized to match GPT-1 ( $L = 12$ ,  $H = 768$ ) purely so the two could be compared fairly, even though their attention patterns (bidirectional vs. causal) are fundamentally different.

## 1.4 The Two-Stage Training Pipeline (p. 57)

**Core idea.** The diagram (from Jay Alammar) shows BERT used in two separate stages.

**Stage 1 – semi-supervised (self-supervised) pretraining.** BERT is trained on massive unlabeled text (books, Wikipedia) with the objective “predict the masked word” (detailed on the next slide). No manual labels are used here; the “labels” are just the original words that were hidden.

**Stage 2 – supervised fine-tuning.** The pretrained BERT weights from Stage 1 are reused as the starting point, and a small new classifier head is attached on top. This whole thing is then trained (fine-tuned) on a small labeled dataset for the actual target task – the diagram’s example is spam classification.

**Mechanism of the classifier head.** BERT produces a special aggregate vector for the whole sequence, called  $C \in \mathbb{R}^H$  (the final hidden state at the first, “[CLS]”, position). The classifier head is just one new linear layer plus a softmax:

$$\text{logits} = CW_{\text{cls}} + b_{\text{cls}}, \quad W_{\text{cls}} \in \mathbb{R}^{H \times 2}, \quad b_{\text{cls}} \in \mathbb{R}^2,$$

followed by softmax to turn the two logits into class probabilities (e.g. 75% Spam, 25% Not Spam in the diagram). Only  $W_{\text{cls}}, b_{\text{cls}}$  are new; everything below them is the already-pretrained BERT.

## 1.5 Pretraining Objectives: Masked LM and Next Sentence Prediction (p. 58)

BERT is pretrained on two self-supervised tasks simultaneously (same forward pass, two loss terms added together).

**Masked Language Modeling (MLM).** 15% of WordPiece token positions in each sequence are chosen at random for prediction. For each chosen position:

- 80% of the time, replace the token with a special [MASK] token.
- 10% of the time, replace it with a random token from the vocabulary.
- 10% of the time, leave it unchanged.

The final hidden vector  $T_i \in \mathbb{R}^H$  at each chosen position is passed through a softmax over the whole vocabulary, and trained with cross-entropy loss against the true (original, un-corrupted) token.

**Why the 80/10/10 split, not just always [MASK]?** This is not justified on the slide, so here is the expansion from the BERT paper (Devlin et al., Section 3.1): if every masked position were always replaced by the literal [MASK] token during pretraining, the model would only ever need to build good representations for [MASK] positions. But at fine-tuning time, [MASK] never appears at all – creating a mismatch between what the model saw in pretraining and what it sees afterward. Mixing in random tokens and unchanged tokens forces BERT to keep building a good contextual representation for *every* token, not just tokens that happen to be masked, which reduces that pretrain/fine-tune mismatch.

**Numerical walkthrough (MLM).** Toy vocabulary of 5 tokens, indices 0–4: {the, cat, sat, on, mat}. Toy hidden size  $d = 4$ . Sequence: “the cat sat”; position 3 (“sat”) is chosen for masking, and happens to fall in the 80% bucket, so the input becomes “the cat [MASK]”.

Suppose the encoder’s output hidden vector at that position is  $T_3 = [0.1, -0.2, 0.4, 0.3]$ . Project through an output matrix (in practice, this is often the transpose of the input embedding matrix, a technique called weight tying)  $W_{\text{out}} \in \mathbb{R}^{4 \times 5}$  to get logits over the vocabulary:

$$\text{logits} = T_3 W_{\text{out}} = [1.2, 0.3, 2.1, 0.5, -0.4].$$

Softmax (subtract the max, 2.1, for numerical stability):

$$[-0.9, -1.8, 0, -1.6, -2.5] \xrightarrow{\text{exp}} [0.407, 0.165, 1.000, 0.202, 0.082], \quad \text{sum} = 1.856.$$

$$P = [0.219, 0.089, 0.539, 0.109, 0.044].$$

The true token is “sat” (index 2), assigned probability 0.539. Cross-entropy loss for this position:

$$L = -\ln(0.539) \approx 0.619.$$

Only this one position’s loss is computed for MLM here; the other, un-masked positions contribute nothing to the MLM loss (though they do still take part in NSP, described next).

**Next Sentence Prediction (NSP).** Input is two packed “sentences”  $A$  and  $B$ : [CLS]  $A$  [SEP]  $B$  [SEP], with a learned segment embedding marking every token as belonging to  $A$  or  $B$ . Training data is constructed so that 50% of the time  $B$  really is the sentence that follows  $A$  in the source text (label *IsNext*), and 50% of the time  $B$  is a random sentence from elsewhere in the corpus (label *NotNext*). The aggregate vector  $C \in \mathbb{R}^H$  (same [CLS] vector used by the classifier head above) is fed into a binary classifier trained to predict *IsNext* vs. *NotNext*. The BERT paper reports this task reaches 97–98% accuracy on its own.

**Why NSP is needed.** Plain language modeling (predicting individual words) never has to reason about the relationship *between* two separate spans of text. Tasks like question answering and natural language inference are fundamentally about that relationship. NSP is a way to give the model that signal for free, without any manual labeling – any two consecutive sentences in a corpus are automatically a positive example.

**Contrast (do not confuse these).** MLM operates at the *token* level (bidirectional context, one loss term per masked token, using each masked position’s own hidden vector  $T_i$ ). NSP operates at the *sentence-pair* level (one loss term per training example, using only the single aggregate  $C$  vector). They are trained jointly, not one after the other – a single forward pass produces both losses, which are summed before backpropagation.

## 1.6 Using a Pretrained BERT: Fine-Tuning vs. Feature Extraction (p. 59)

**Core idea.** Once BERT is pretrained, there are two different ways to use it for a downstream task.

- **Fine-tuning.** Add a small task-specific layer (as in the classifier-head example above) and update BERT’s own weights together with the new layer, using the labeled downstream data.
- **Feature extraction.** Freeze BERT entirely. Run it once over your data, pull out its hidden states as fixed numeric features, and train a separate (usually much smaller) model on top of those frozen features. BERT itself is never updated.

**Mechanism – which hidden states to use?** BERT-BASE produces, for every token, one  $H$ -dimensional ( $H = 768$ ) hidden vector per layer: the input embedding, plus one output per encoder block, i.e. 13 vectors total per token for a 12-layer model. If you are doing feature extraction, you must choose which of these 13 vectors (or combination of them) to use. The slide’s table (from Jay Alammur, reproducing an ablation from the BERT paper, Section 5.3, on the CoNLL-2003 Named Entity Recognition task) compares choices, each measured by dev-set F1:

| Feature choice                           | Dev F1 |
|------------------------------------------|--------|
| First layer (embeddings) only            | 91.0   |
| Last hidden layer only                   | 94.9   |
| Weighted sum of all 12 layers            | 95.5   |
| Second-to-last hidden layer              | 95.6   |
| Weighted sum of last four hidden layers  | 95.9   |
| Concatenation of last four hidden layers | 96.1   |

(The BERT paper’s own labels say “weighted sum,” which the lecture slide shortens to just “sum”; the numbers match exactly.)

**Dimension detail.** “Sum last four hidden” adds four 768-dimensional vectors elementwise, so the resulting feature vector is still 768-dimensional. “Concat last four hidden” instead stacks them side by side, giving a  $4 \times 768 = 3072$ -dimensional feature vector – more information is preserved, which is presumably why it scores best (96.1, only 0.3 F1 behind full fine-tuning at 96.4 for BASE).

**What happens to these features?** The BERT paper specifies exactly this (Section 5.3): the frozen features are fed into a newly-initialized, two-layer, 768-dimensional BiLSTM, followed by a classification layer over the NER tag set. This downstream model, not BERT, is what gets trained in the feature-extraction setting.

**Contrast (do not confuse these).** Fine-tuning updates BERT’s own weights and typically gets the best accuracy, at the cost of needing to backpropagate through the whole model for every new task. Feature extraction never touches BERT’s weights – you compute BERT’s outputs once, cache them, and can then cheaply train many small downstream models (for many different tasks) on top of the same cached features, without ever re-running BERT itself.

## 2 Decoder-Only Language Models and Modern LLMs (Lecture 5, pp. 3–31)

### 2.1 Decoder-Only Generative Models (p. 3)

**Core idea.** Most modern generative language models (GPT, LLaMA, and similar) use only the *decoder* half of the original transformer, and are trained with causal language modeling: predict the next token given only the tokens that came before it, never the ones after.

**Mechanism.** “Causal” is enforced by masking inside self-attention: position  $i$  is only allowed to attend to positions  $\leq i$ . Concretely, before the softmax in attention, all scores for positions  $j > i$  are set to  $-\infty$ , so after softmax those positions get weight 0.

**Where this architecture comes from.** The slide cites Sections 4.2.3 and 4.2.4 of a specific paper. That paper is Liu et al., *Generating Wikipedia by Summarizing Long Sequences* (2018) – it is the paper that first proposed dropping the encoder entirely and training a single decoder stack as an ordinary language model. Section 4.2.3 (“Transformer Decoder”) describes exactly this: concatenate an input sequence and a target sequence into one long sequence  $(w_1, \dots, w_{n+\eta})$  (separated by a delimiter token), and train it to predict each next token from everything before it:

$$p(w_1, \dots, w_{n+\eta}) = \prod_{j=1}^{n+\eta} p(w_j \mid w_1, \dots, w_{j-1}).$$

This factorization – the joint probability of a whole sequence written as a product of next-token conditionals – is the mathematical core of every decoder-only LM discussed in the rest of this section.

**Contrast (do not confuse these).** BERT’s self-attention (Part 1) is bidirectional: no masking, every position can see every other position. A decoder-only model’s self-attention is causal/masked: position  $i$  only sees positions  $\leq i$ . Both are still “self”-attention (queries, keys, and values all come from the same sequence) – the only difference is the mask. This is different again from *cross*-attention, which appears only in encoder–decoder models and is discussed on p. 8 below.

### 2.2 The Next-Token Prediction Objective (p. 4)

**Core idea.** The slide’s own example: input “The capital of Bangladesh is”, target “Dhaka”. At every position in a training sequence, the model predicts a probability distribution over the vocabulary for the *next* token, and is scored against the actual next token in the training text.

**Mechanism, term by term.**

- *Cross-entropy loss*: same idea as MLM’s loss above – compare the predicted distribution to a one-hot vector at the true next token,  $L = -\log P(\text{correct next token})$ .
- *Teacher forcing*: during training, the input fed into the model at every position is always the true, ground-truth previous token – never the model’s own (possibly wrong) earlier prediction. This means every position’s loss can be computed independently and in parallel in a single forward pass, since the causal mask already prevents position  $i$  from seeing  $j > i$ , and the correct token for position  $j$  is simply the training text’s own token  $j$ , always available.

- *Autoregressive generation* (inference time): there is no ground truth to feed in anymore, so the model must feed its own previously generated tokens back in as the next input. This creates a training/inference mismatch (often called *exposure bias*): any early mistake at inference can compound, since nothing forces the model to see mistaken inputs during training the way it will at inference.

**Numerical walkthrough.** Toy vocabulary of 5 tokens, indices 0–4: {the, cat, sat, on, mat}. Sequence  $x = (\text{the}, \text{cat}, \text{sat})$ . Because of causal masking, the model produces *three* next-token predictions in one forward pass: position 1 predicts token 2 (“cat”), position 2 predicts token 3 (“sat”), position 3 predicts whatever comes next in the training text (say, “on”). Take position 3: suppose its output hidden vector is  $h_3 = [0.2, 0.5, -0.1, 0.3]$ , projected through an output matrix  $W_{\text{out}} \in \mathbb{R}^{4 \times 5}$  (often tied to the input embedding matrix, to save parameters) to logits

$$\text{logits} = [0.9, 0.2, 0.1, 2.0, 0.4],$$

where index 3 (“on”) is the true target. Softmax (subtract max 2.0):

$$[-1.1, -1.8, -1.9, 0, -1.6] \xrightarrow{\text{exp}} [0.333, 0.165, 0.150, 1.000, 0.202], \quad \text{sum} = 1.850.$$

$$P = [0.180, 0.089, 0.081, 0.541, 0.109].$$

True token “on” gets probability 0.541, giving loss  $L = -\ln(0.541) \approx 0.614$ . Positions 1 and 2 each contribute their own such loss term the same way; all three are averaged (or summed) into the batch loss.

**Contrast (do not confuse these).** MLM (BERT) predicts only the  $\sim 15\%$  of positions that were masked, using bidirectional context. Causal LM (GPT-style) predicts *every* position’s next token, using only leftward context. This means a causal LM gets a much denser training signal per token processed – one loss term per position instead of one per masked position – which is one practical reason decoder-only causal LMs became the dominant recipe for pure text generation.

## 2.3 Tokenization Recap (p. 5)

Three subword tokenization schemes are named: Byte Pair Encoding / byte-level BPE (GPT-2, GPT-3, RoBERTa), WordPiece (BERT), and SentencePiece (T5, LLaMA). The slide marks this as already covered earlier in the course, so it is not re-derived here – the only new information on this slide is which model family uses which scheme.

## 2.4 Pretraining Data Sources (p. 6)

Modern LLM pretraining corpora are usually a mixture of: Common Crawl (petabyte-scale, noisy web text), books (Project Gutenberg and similar), Wikipedia (clean and structured), academic papers (ArXiv, PubMed), and code (GitHub). The slide’s stated principle: data *quality* matters more than raw *quantity*. Open issues it flags without resolving: diversity of sources, quality control, and contamination (test-set text leaking into the training corpus, inflating benchmark scores).

## 2.5 Scaling Laws and Chinchilla (p. 7)

**Core idea.** Many early large models were “compute-inefficient”: too large for the amount of data they were trained on. The slide’s claim, from Hoffmann et al. (2022), the “Chinchilla” paper: model size and training-data size should be scaled up *together*, roughly equally – doubling one without doubling the other wastes compute. (Note: the slide’s citation reads “Hoffman et al.”; the paper’s actual first author is Hoffmann, with two n’s.)

**Mechanism – where the trade-off comes from.** The paper approximates the training compute (in FLOPs) of a transformer as

$$C \approx 6 N D,$$

where  $N$  is the parameter count and  $D$  is the number of training tokens (this approximation, attributed in the paper to Kaplan et al. 2020, accounts for roughly 2 FLOPs per parameter per token in the forward pass, times about 3 for a combined forward-plus-backward pass). For any fixed compute budget  $C$ , this ties  $N$  and  $D$  together: spend the budget on a bigger model, and you can afford fewer tokens; spend it on more tokens, and you must use a smaller model. The paper’s empirical finding is that the compute-optimal split is to scale  $N$  and  $D$  in *equal proportion* as  $C$  grows – contradicting an earlier estimate (Kaplan et al.) that model size should grow much faster than data.

**The headline comparison.** Gopher:  $N = 280\text{B}$  parameters,  $D = 300\text{B}$  tokens. Chinchilla:  $N = 70\text{B}$  parameters ( $4\times$  smaller), trained on  $D = 1.4\text{T}$  tokens ( $4\times$  more), using approximately the *same* total training compute as Gopher – and it outperforms Gopher across a wide range of benchmarks.

**Numerical sanity check using  $C \approx 6ND$ .** Gopher’s implied compute:

$$C_{\text{Gopher}} \approx 6 \times (280 \times 10^9) \times (300 \times 10^9) = 5.04 \times 10^{23} \text{ FLOPs},$$

close to the paper’s reported figure of  $5.76 \times 10^{23}$  FLOPs for Gopher (the small gap is expected, since 280B/300B are rounded figures and  $6ND$  is itself only an approximation, not an exact accounting). Holding that same compute budget fixed and solving for the token count a 70B-parameter model could be trained on:

$$D = \frac{C}{6N} = \frac{5.76 \times 10^{23}}{6 \times (70 \times 10^9)} \approx 1.37 \times 10^{12} \text{ tokens} \approx 1.37 \text{ trillion tokens},$$

which matches almost exactly the paper’s actual reported figure for Chinchilla, 1.4 trillion tokens – confirming the  $6ND$  approximation and the “ $4\times$  smaller,  $4\times$  more data, same compute” story are mutually consistent.

## 2.6 GPT’s Decoder-Only Architecture (p. 8)

**Core idea.** The slide diagram places the original encoder–decoder transformer (left) next to a GPT-style decoder-only stack (right, labelled “ $12\times$ ” for a 12-block model, e.g. GPT-1). The right-hand stack keeps only masked multi-head self-attention and a feed-forward block per layer, repeated  $12\times$ , sitting directly on top of a combined text-and-position embedding, and feeding a shared output head (“Text Prediction” / “Task Classifier”).

**Mechanism – what got removed.** The original transformer’s decoder block has *three* sublayers: masked self-attention, then cross-attention (attending to the encoder’s output), then a feed-forward layer. A decoder-only model has no encoder to attend to, so the cross-attention sublayer is simply dropped – each block keeps only masked self-attention and the feed-forward layer.

**Contrast (do not confuse these): masked self-attention vs. cross-attention.**

- *Masked self-attention*: queries, keys, and values all come from the *same* sequence (the decoder’s own tokens so far); a causal mask blocks attending to future positions.
- *Cross-attention* (only present in encoder–decoder models, absent from decoder-only models like GPT): queries come from the decoder’s current sequence, but keys and values come from the *encoder’s* output. This lets the decoder look at the full (unmasked) source sequence while generating the target sequence.

GPT, having no encoder, has no cross-attention at all – every attention operation in a GPT-style model is masked self-attention.

**Feed-forward shapes.** Each block’s feed-forward sublayer expands then contracts:

$$[\text{seq\_len}, H] \xrightarrow{\text{Linear}} [\text{seq\_len}, 4H] \xrightarrow{\text{nonlinearity}} [\text{seq\_len}, 4H] \xrightarrow{\text{Linear}} [\text{seq\_len}, H].$$

Ignoring biases, that is  $H \times 4H + 4H \times H = 8H^2$  parameters per block. For  $H = 768$  (GPT-1/BERT-BASE scale):  $8 \times 768^2 \approx 4.72\text{M}$  per block,  $\times 12$  blocks  $\approx 56.6\text{M}$  just from feed-forward layers – consistent with GPT-1’s total of roughly 120M parameters once attention and embedding weights are added in.

## 2.7 GPT-1: One Pretrained Model, Many Tasks via Input Reformatting (p. 9)

**Core idea.** GPT-1 is adapted to a downstream task without changing its architecture at all – only the *input formatting* changes, using special tokens **Start**, **Delim**, **Extract**, plus one new linear layer read from the final (**Extract**) position’s hidden state.

**Mechanism, per task type (from the slide’s diagram).**

- *Classification*: **Start** Text **Extract**  $\rightarrow$  Transformer  $\rightarrow$  Linear.
- *Entailment*: **Start** Premise **Delim** Hypothesis **Extract**  $\rightarrow$  Transformer  $\rightarrow$  Linear.
- *Similarity*: run *both* orderings, (Text1, Text2) and (Text2, Text1), through the same Transformer separately, *add* the two resulting representations, then apply one Linear layer. The addition is necessary because the model is causal (order-sensitive), but similarity between two texts should not depend on which one is listed first.
- *Multiple choice*: build one full sequence per candidate answer (**Start** Context **Delim** Answer<sub>*i*</sub> **Extract**), run each through the same Transformer + Linear to get one scalar score per answer, then softmax across all candidate scores to choose the most likely one.

**Why this works.** Exactly like BERT’s “one additional output layer” pattern (p. 56/57 above), GPT-1 keeps a single pretrained Transformer stack and only ever adds one small new Linear layer



per task – the expensive pretraining is done once, and adapting to a new task is cheap. This diagram comes directly from the lecture slide; the GPT-1 paper itself is not among the reference files provided for this session, so this description is grounded only in the slide’s own figure, not independently re-verified against the original paper text.

## 2.8 GPT-2 (p. 10)

GPT-2 is described as a “direct scale-up” of GPT-1: roughly a ten-fold increase in both parameter count and training-data size. Concretely: GPT-1  $\approx$  120M parameters, GPT-2 = 1.5B parameters. Trained on WebText (pages linked from Reddit posts with at least 3 karma, before December 2017 – about 45 million links, 40GB of text). It was the last GPT model OpenAI released with public (downloadable) weights.

*Small note:* 1.5B/120M  $\approx$  12.5, somewhat more than a literal “ten-fold” increase – the “ten-fold” phrasing (from the original GPT-2 paper) is a rounded description, not an exact ratio; the two given parameter counts are what should be trusted for arithmetic.

## 2.9 GPT-3 (p. 11)

A further scale-up: 175B parameters (largest variant), trained on about 300 billion tokens drawn from a mixture of Common Crawl, WebText, English Wikipedia, and two book corpora (570GB of raw text). Unlike GPT-2, it was never released with public weights – access is only through an API. The slide marks its own cost figures (355 GPU-years, \$4.6 million) with a question mark, meaning the lecture itself is not fully certain of them; they are reproduced here with that same uncertainty rather than stated as settled facts. GPT-3.5 (an instruction-tuned descendant of GPT-3) is credited with triggering the current wave of LLM adoption.

## 2.10 GPT-4 and Beyond (p. 12)

Briefly: multimodal inputs, instruction-tuning combined with reinforcement learning from human and AI feedback, larger context windows (up to 32k tokens in the version described), and improved chat behavior. The slide does not go into architectural detail here, so neither do these notes.

## 2.11 The LLaMA Model Family (p. 13)

**Core idea.** LLaMA (Meta AI) is one of the most widely used openly-available LLM families, with three generations and several parameter sizes per generation.

| Model   | Training data       | Params | Context | GQA | Tokens | LR                   |
|---------|---------------------|--------|---------|-----|--------|----------------------|
| LLaMA 1 | Touvron et al. 2023 | 7B     | 2k      | ×   | 1.0T   | $3.0 \times 10^{-4}$ |
|         |                     | 13B    | 2k      | ×   | 1.0T   | $3.0 \times 10^{-4}$ |
|         |                     | 33B    | 2k      | ×   | 1.4T   | $1.5 \times 10^{-4}$ |
|         |                     | 65B    | 2k      | ×   | 1.4T   | $1.5 \times 10^{-4}$ |
| LLaMA 2 | New public data mix | 7B     | 4k      | ×   | 2.0T   | $3.0 \times 10^{-4}$ |
|         |                     | 13B    | 4k      | ×   | 2.0T   | $3.0 \times 10^{-4}$ |
|         |                     | 34B    | 4k      | ✓   | 2.0T   | $1.5 \times 10^{-4}$ |
|         |                     | 70B    | 4k      | ✓   | 2.0T   | $1.5 \times 10^{-4}$ |

**Reading the table.** Two patterns worth noticing: (1) learning rate decreases as model size increases within each generation (larger models need smaller updates for stable training); (2) LLaMA 2 doubles the context window (2k→4k) and roughly doubles training tokens (up to 1.4T→2.0T) relative to LLaMA 1.

**Important flag: Grouped-Query Attention (GQA) is a LLaMA-2-only feature, and only for the two largest sizes.** This table is explicit: GQA is × (not used) for every LLaMA-1 size and for LLaMA-2 7B/13B, and only ✓ (used) for LLaMA-2 34B and 70B. This directly contradicts the next slide’s architecture diagram, which is labelled “LLaMA 1 Architecture” but depicts Grouped Multi-Query Attention as part of it, and the slide after that, which lists Grouped Multi-Query Attention under “LLaMA 1 new concepts.” Going by this table (the most precise, tabular source of the three), GQA should really be attributed to LLaMA 2’s largest variants, not to LLaMA 1 in general. This inconsistency in the lecture material is not resolved by anything else in the slides – it is flagged here rather than silently repeated.

## 2.12 LLaMA Architecture Diagram (p. 14)

**Core idea.** The diagram contrasts the original transformer’s decoder stack (left) with a generic “LLaMA architecture” stack (right). Reading the right-hand stack bottom to top, repeated  $N \times$ : embeddings; RMSNorm; self-attention (labelled “Grouped Multi-Query Attention with KV Cache,” with rotary position encodings applied inside the attention); residual add; RMSNorm; feed-forward using a SwiGLU activation; residual add. After the last repeat: a final RMSNorm, then a Linear layer, then Softmax.

Each new component gets its own dedicated slide later (RoPE on p. 16–17, GQA on p. 18); this slide is an overview. Two components – RMSNorm and SwiGLU – are not otherwise explained anywhere in the lecture slides or in any of the three reference papers provided for this session (BERT, the decoder-only paper, and the Chinchilla paper); the definitions below come from general, well-established background on these two techniques (Zhang & Sennrich for RMSNorm; Shazeer for SwiGLU), not from a paper uploaded to this project, and are noted as such.

**RMSNorm.** Standard LayerNorm centers and rescales a vector using both its mean and its standard deviation, plus a learned scale *and* a learned bias. RMSNorm skips the centering step (no mean subtraction) and uses only the root-mean-square:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{k=1}^d x_k^2 + \epsilon}} \odot g, \quad g \in \mathbb{R}^d \text{ learned, } x \in \mathbb{R}^d.$$

This is cheaper to compute (no mean, no separate bias vector) and is used in place of LayerNorm throughout the LLaMA family.

*Toy example:*  $x = [3, 4, 0, 0]$  ( $d = 4$ ),  $g = [1, 1, 1, 1]$ ,  $\epsilon \approx 0$ .  $\sqrt{\frac{1}{4}(9 + 16 + 0 + 0)} = \sqrt{6.25} = 2.5$ .  $\text{RMSNorm}(x) = [3/2.5, 4/2.5, 0, 0] = [1.2, 1.6, 0, 0]$ .

**SwiGLU.** Replaces a plain feed-forward nonlinearity (e.g. ReLU) with a *gated* one: two separate linear projections of the input, one of them passed through a Swish/SiLU nonlinearity, then multiplied together elementwise before the final down-projection:

$$\text{SwiGLU}(x) = (\text{Swish}(xW)) \odot (xV), \quad \text{Swish}(z) = z \cdot \sigma(z),$$

with  $W, V \in \mathbb{R}^{H \times d_{ff}}$  two independent projections of the  $H$ -dimensional input, and  $\odot$  elementwise multiplication – so the “gate”  $\text{Swish}(xW)$  decides how much of  $xV$  passes through, per dimension.

## 2.13 LLaMA 1 New Concepts (p. 15)

This slide is a table of contents for the four new ideas introduced in the LLaMA architecture: normalization (RMSNorm, above), Rotary Positional Embeddings, Grouped Multi-Query Attention, and the SwiGLU activation function (above). See the flag on p. 13 regarding which LLaMA generation actually introduces Grouped Multi-Query Attention.

## 2.14 Relative Positional Encoding (Shaw et al.) (p. 16)

**Core idea.** Instead of encoding a token’s *absolute* position by adding a fixed or learned position vector to its embedding (as the original transformer does, once, before any attention), this method adds a learned bias directly into the attention computation, based on the *relative* distance between the query position and the key position.

**Mechanism.** For query position  $i$  and key position  $j$ :

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}, \quad \alpha_{ij} = \text{softmax}_j(e_{ij}), \quad z_i = \sum_j \alpha_{ij} (x_j W^V + a_{ij}^V).$$

Here  $x_i, x_j \in \mathbb{R}^{d_{\text{model}}}$  are token representations;  $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_z}$  are the ordinary query/key/value projections ( $d_z$  is the per-head dimension, the same idea as  $d_k$  in the more familiar notation); and  $a_{ij}^K, a_{ij}^V \in \mathbb{R}^{d_z}$  are *learned* vectors indexed only by the (clipped) relative offset  $j - i$ , not by  $i$  and  $j$  individually – so any two position-pairs with the same offset (e.g. query-attends-one-step-back, anywhere in the sequence) reuse the exact same learned vector. In practice the offset is clipped to some maximum  $\pm k$ , so only  $2k + 1$  distinct vectors ever need to be learned, no matter how long the sequence is.

**Numerical walkthrough.** Toy  $d_z = 2$ . Sequence “the cat sat,” query at position  $i = 3$  (“sat”), attending over keys at  $j = 1, 2, 3$ . Already-projected (toy) keys:  $k_1 = [1, 0]$ ,  $k_2 = [0, 1]$ ,  $k_3 = [1, 1]$ ; query  $q_3 = [1, 1]$ . Learned relative-key biases by offset  $j - i$ :  $a_{-2}^K = [0.1, 0.1]$ ,  $a_{-1}^K = [0.2, -0.1]$ ,  $a_0^K = [0, 0]$ .

$$e_{3,1} = \frac{q_3 \cdot (k_1 + a_{-2}^K)}{\sqrt{2}} = \frac{[1, 1] \cdot [1.1, 0.1]}{\sqrt{2}} = \frac{1.2}{1.4142} = 0.849,$$

$$e_{3,2} = \frac{[1, 1] \cdot [0.2, 0.9]}{\sqrt{2}} = \frac{1.1}{1.4142} = 0.778, \quad e_{3,3} = \frac{[1, 1] \cdot [1, 1]}{\sqrt{2}} = \frac{2}{1.4142} = 1.414.$$

Softmax (subtract max 1.414):  $[-0.566, -0.636, 0] \xrightarrow{\text{exp}} [0.568, 0.529, 1.000]$ , sum = 2.097, giving

$$\alpha_{3,\cdot} = [0.271, 0.252, 0.477].$$

With toy values  $v_1 = [0.5, 0.5]$ ,  $v_2 = [1, 0]$ ,  $v_3 = [0, 1]$  and relative-value biases  $a_{-2}^V = [0, 0.1]$ ,  $a_{-1}^V = [0.1, 0]$ ,  $a_0^V = [0, 0]$ :

$$z_3 = 0.271[0.5, 0.6] + 0.252[1.1, 0] + 0.477[0, 1] = [0.413, 0.639].$$

This shows the full mechanism: the relative-offset bias enters *both* the raw attention score (through the key) and the final weighted output (through the value), and the same learned vector is reused for every pair of positions sharing that offset.

## 2.15 Rotary Positional Encoding (RoPE) (p. 17)

**Core idea.** Instead of adding a position signal to the embeddings (absolute, additive), or adding a learned bias per relative offset inside attention (Shaw et al., above), RoPE *rotates* the query and key vectors by an angle proportional to their absolute position. Because rotation is an orthogonal operation, the dot product between a query rotated for position  $m$  and a key rotated for position  $n$  depends only on  $(m - n)$  – the relative offset – even though each was rotated using only its own absolute position. Relative-position-aware attention falls out “for free” from a purely absolute-position operation.

**Mechanism (2D case, then general  $d$ ).** For a 2-dimensional sub-block,

$$f_{\{q,k\}}(x_m, m) = R(m\theta) W_{\{q,k\}} x_m, \quad R(m\theta) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix},$$

i.e. project  $x_m \in \mathbb{R}^2$  with the ordinary  $W_{\{q,k\}} \in \mathbb{R}^{2 \times 2}$ , then rotate the result by angle  $m\theta$ . For a general even dimension  $d$ , the same idea is applied to  $d/2$  independent 2-D pairs of coordinates  $(1, 2), (3, 4), \dots$ , each pair rotated by its own angle  $m\theta_t$  (the  $\theta_t$  form a geometrically decreasing sequence across pairs, the same idea as the frequency schedule in the original sinusoidal encoding), giving a block-diagonal rotation matrix  $R_{\Theta, m}^d \in \mathbb{R}^{d \times d}$  so that  $f_{\{q,k\}}(x_m, m) = R_{\Theta, m}^d W_{\{q,k\}} x_m \in \mathbb{R}^d$  – same shape in and out, only the direction of the vector changes. The slide’s bottom formula is an efficient, elementwise way to compute this rotation pair-by-pair, without ever building the full  $d \times d$  matrix –  $O(d)$  elementwise multiplications instead of an  $O(d^2)$  matrix multiply.

**Why relative position falls out automatically.** For rotation matrices,  $R(a)^T = R(-a)$ , and  $R(-a)R(b) = R(b - a)$ . So for a query rotated at position  $m$  and a key rotated at position  $n$  (both built from the same un-rotated vectors  $q, k$ ):

$$q_m \cdot k_n = q^T R(m\theta)^T R(n\theta) k = q^T R((n - m)\theta) k,$$

which depends only on  $n - m$ .

**Numerical walkthrough.** Let  $\theta = 90^\circ$ , and un-rotated toy vectors  $q = k = [1, 0]$ .

- $m = 1, n = 3$  (offset = 2):  $q_1 = R(90^\circ)[1, 0] = [0, 1]$ ;  $R(270^\circ) = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ , so  $k_3 = R(270^\circ)[1, 0] = [0, -1]$ . Dot product:  $0 \cdot 0 + 1 \cdot (-1) = -1$ .
- $m = 6, n = 8$  (offset = 2, same as above):  $R(6 \times 90^\circ) = R(540^\circ) = R(180^\circ) = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}$ , so  $q_6 = [-1, 0]$ ;  $R(8 \times 90^\circ) = R(720^\circ) = R(0^\circ) = I$ , so  $k_8 = [1, 0]$ . Dot product:  $(-1)(1) + (0)(0) = -1$ .

Both position pairs give the same dot product  $(-1)$  because both share the same relative offset (2), even though the absolute positions are completely different – a direct numerical confirmation of the relative-position property.

**Contrast (do not confuse these).** Shaw et al.’s relative encoding (p. 16) *adds* a small *learned* vector to keys/values, indexed by a *clipped* relative offset – it needs  $2k + 1$  trained parameters and cannot distinguish offsets beyond  $\pm k$ . RoPE instead *multiplies* (rotates)  $Q$  and  $K$  by a fixed, *not learned*, deterministic function of *absolute* position, and the relative-position behaviour falls out of the geometry with no extra parameters and no clipping – it generalizes cleanly to sequence lengths never seen during training, which the clipped, learned table in Shaw et al. cannot do.

## 2.16 Grouped Multi-Query Attention (GQA) (p. 18)

**Core idea.** Reduces how much Key/Value data must be moved through memory during attention (not how much compute is needed), by having several query heads share the same K/V head, instead of every query head owning a private K/V pair.

**Mechanism.** With  $h$  query heads:

- *Multi-Head Attention (MHA)*:  $h$  separate K heads and  $h$  separate V heads, one uniquely paired to each Q head.
- *Grouped-Query Attention (GQA)*: the  $h$  Q heads are split into  $g < h$  groups; only  $g$  distinct K heads and  $g$  distinct V heads exist, each shared by  $h/g$  Q heads.
- *Multi-Query Attention (MQA)*: the extreme case  $g = 1$  – every Q head shares one single K head and one single V head.

**Why this matters.** During autoregressive decoding, the bottleneck is usually not the arithmetic of attention (GPUs do that very fast) but the memory bandwidth needed to reload the cached K/V tensors (the “KV cache”) at every single decoding step. Fewer distinct K/V heads directly means less K/V data to read per step, while all  $h$  Q heads – and therefore the model’s full representational capacity on the query side – are kept.

**Numerical illustration.** Suppose  $h = 8$  query heads, each of dimension  $d_{\text{head}} = 64$  (matching the diagram’s 8-head example).

- Full MHA: K cache per token =  $8 \times 64 = 512$  values, same for V: 1024 total per token.

- GQA with  $g = 4$  groups (2 Q heads per group, as in the diagram): K cache per token =  $4 \times 64 = 256$ , V the same: 512 total per token – half the memory traffic of full MHA, with all 8 Q heads intact.
- MQA ( $g = 1$ ): K cache per token =  $1 \times 64 = 64$ , V the same: 128 total per token – an  $8\times$  reduction versus full MHA.

**Contrast.** The only thing GQA changes relative to ordinary multi-head attention (already covered earlier in the course) is how many independent K/V projections exist relative to Q projections. The causal masking, the softmax, and the value-weighted sum are all otherwise identical.

## 2.17 LLaMA 2’s Scale (p. 19)

LLaMA 2 70B, at release, was one of the largest openly-available LLMs: trained on 10TB of text, needing about 6,000 GPUs running for 12 days, roughly  $10^{24}$  floating-point operations, and about \$2 million. (This “10TB of text” figure and the “2.0T tokens” figure from the table on p. 13 are not in conflict – they are different units, raw bytes of text versus tokenized count, and a few bytes per token is a normal ratio.)

## 2.18 Long Context Models (p. 20)

**Core idea.** A model’s context window is the number of tokens it can consider at once. This has grown quickly across generations: 4K (GPT-3)  $\rightarrow$  128K (GPT-4 Turbo)  $\rightarrow$  1M+ (Gemini 1.5) – enabling whole-book analysis, full-codebase reasoning, and long legal contracts in a single pass.

**Limitations named on the slide.** (1) The “lost in the middle” phenomenon: models tend to use information from the very start and very end of a long context better than information buried in the middle. (2) Quadratic attention cost: standard self-attention computes a score for every pair of positions, so its cost scales as  $O(N^2)$  in sequence length  $N$ .

**Numerical illustration of the quadratic cost.** At  $N = 4K$  tokens, the attention score matrix has  $4096^2 \approx 16.8$  million entries per head. At  $N = 128K$  ( $32\times$  longer), it has  $128\,000^2 \approx 16.4$  *billion* entries per head – about  $32^2 = 1024\times$  more entries for only a  $32\times$  longer sequence. FlashAttention (mentioned as a mitigation) is explicitly marked “not covered” on the slide, so it is not detailed further here.

## 2.19 Mixture of Experts (MoE) (p. 21)

**Core idea.** Instead of every token passing through one shared feed-forward block, an MoE layer has many parallel feed-forward “expert” sub-networks, and a lightweight router selects only a small number of them (top- $k$ ) to actually process each token.

**Mechanism.** Per the slide, only the feed-forward sublayers are replicated into experts – token embeddings and attention layers remain shared, single copies, exactly as in a dense transformer. Mixtral  $8\times 7B$  has 8 experts; the total parameter count is 47B, *not* a naive  $8 \times 7 = 56B$ , precisely because only the FFN portion is duplicated eight times while attention and embeddings are shared once. At inference, only 2 of the 8 experts are active per token, giving 13B “active” parameters

actually used per forward pass, even though the model occupies memory as if it were 47B. DeepSeek-V3 uses a much finer split: 256 experts total, 8 active per token.

## 2.20 Multimodal Architectures (p. 22)

Explicitly marked “not covered in this course” on the slide; noted here for completeness, not expanded.

## 2.21 Faster, Cheaper, Deployable Models: Overview (p. 23)

Table-of-contents slide for the three efficiency techniques covered next: quantization, parameter-efficient fine-tuning (PEFT), and pruning/distillation.

## 2.22 Quantization: Concept (p. 24)

**Core idea.** Quantization represents weights and activations with fewer bits (e.g. 8-bit integers instead of 32-bit floats), shrinking memory footprint, cutting energy use, and speeding up matrix multiplication (integer arithmetic at a given bit width is typically cheaper on hardware than float arithmetic, and fewer bits per number means less data movement).

**Two flavors.** *Post-training quantization*: quantize an already fully-trained model afterward, with no further training – fast, but accuracy can drop since the model never adapted to the lower precision. *Quantization-aware training*: simulate quantization during training itself – the forward pass uses (simulated) quantized values so the model can adapt, while the backward pass still computes gradients in full precision.

**Underspecified in the lecture material.** Neither the slide nor any of the three reference papers provided for this session (BERT, the decoder-only paper, Chinchilla) explain *how* gradients are computed through the forward pass’s quantization step, given that rounding to a fixed grid of values is not a smoothly differentiable operation. (In practice this typically uses a “straight-through” gradient estimator, but since that technique is not covered anywhere in the provided materials, it is flagged here as a genuine gap rather than filled in from outside sources.)

## 2.23 Quantization: Numeric Formats (p. 25)

**Core idea.** Common numeric formats trade off *range* (how large or small a value can be, set by the number of exponent bits) against *precision* (how many significant digits, set by the number of mantissa bits). All are built as: 1 sign bit + exponent bits + mantissa bits (except pure integer types, which have no exponent/mantissa split).

| Format                           | Sign | Exponent | Mantissa |
|----------------------------------|------|----------|----------|
| FP32                             | 1    | 8        | 23       |
| TF32 (used internally in matmul) | 1    | 8        | 10       |
| FP16                             | 1    | 5        | 10       |
| BFLOAT16                         | 1    | 8        | 7        |

**Why this matters mechanically.** FP16 has only 5 exponent bits (same as nothing wider), giving it a *narrower range* than FP32 – its smallest representable positive normal value is around  $2^{-14} \approx 6 \times 10^{-5}$ , which is exactly why LayerNorm’s tiny epsilon ( $\sim 10^{-12}$ ) can silently underflow to zero or cause NaNs in float16 (this is the exact problem the next slide raises). BFLOAT16 keeps FP32’s full 8 exponent bits (same range as FP32) but only 7 mantissa bits (coarser precision than FP16) – it trades precision for keeping FP32’s wide range, avoiding FP16’s overflow/underflow problems.

**Accumulation types.** When adding many low-precision numbers, the running sum needs a wider type to avoid overflow. Slide’s own example, verified: signed int8’s maximum value is  $2^7 - 1 = 127$ . Two int8 values  $A = 127$ ,  $B = 127$  summed give  $C = 254$ , which exceeds 127 and cannot be represented in int8 – hence the sum must be accumulated in a wider type (int32) even though the inputs are int8.

## 2.24 FP32 → FP16 Quantization (p. 26)

Converting FP32 to FP16 is comparatively simple because both are IEEE floating-point formats with the same overall structure, differing only in how many exponent/mantissa bits each uses. Two practical checks matter: (1) does the hardware actually compute in FP16 natively, or only store it while silently upconverting to FP32 for the math (an issue on some older CPUs, resolved on newer ones like Intel’s Cooper Lake/Sapphire Rapids)? (2) is the specific operation sensitive to FP16’s narrower range/precision – as with LayerNorm’s epsilon, discussed above?

## 2.25 FP32 → int8 Quantization: the Affine Scheme (p. 27)

**Core idea.** int8 has only 256 distinct values, far fewer than FP32’s huge range, so a chosen float range  $[a, b]$  must be mapped onto those 256 integer codes.

**Mechanism.**

$$x = S(x_q - Z) \quad (\text{dequantize}), \quad x_q = \text{round}\left(\frac{x}{S} + Z\right) \quad (\text{quantize}),$$

where  $x \in [a, b]$  is the original float value,  $x_q$  is the stored int8 code,  $S > 0$  is the *scale* (a float32 – the real-valued size of one integer step), and  $Z$  is the *zero-point* – the integer code that represents the real value 0 exactly, important because 0 appears constantly throughout ML models (padding, ReLU outputs, and so on) and must be representable without error.

**Numerical walkthrough.** Suppose a weight tensor’s observed range is  $[a, b] = [-2.0, 6.0]$ , mapped onto the 256 codes  $0, \dots, 255$ . Scale:

$$S = \frac{b - a}{255} = \frac{8.0}{255} \approx 0.03137.$$

Zero-point (chosen so real 0 lands exactly on an integer code):

$$Z = \text{round}(-a/S) = \text{round}(2.0/0.03137) = \text{round}(63.75) = 64.$$

Check the two endpoints quantize to the two ends of the code range:

$$x_q(a) = \text{round}(-63.75 + 64) = \text{round}(0.25) = 0,$$



$$x_q(b) = \text{round}(191.25 + 64) = \text{round}(255.25) = 255.$$

Both land exactly at the ends of the 0–255 range, as expected.

Check dequantization recovers the originals (up to small rounding loss, which is the whole point of quantization being lossy):  $x_q = 0 \Rightarrow x = 0.03137 \times (0 - 64) = -2.008 \approx -2.0$ ;  $x_q = 255 \Rightarrow x = 0.03137 \times (255 - 64) = 5.992 \approx 6.0$ ; and the zero-point itself,  $x_q = 64 \Rightarrow x = 0.03137 \times (64 - 64) = 0$ , exactly.

## 2.26 Parameter-Efficient Fine-Tuning (PEFT) (p. 28)

**Core idea.** Instead of updating every weight in a huge pretrained model for each new task (expensive, and requires a full separate copy of the model per task), PEFT freezes almost all of the original weights and trains only a small number of new, task-specific parameters.

**Two named techniques.**

- *Adapters*: small bottleneck feed-forward layers inserted between existing transformer sublayers (down-project  $H \rightarrow r$ , nonlinearity, up-project  $r \rightarrow H$ , with  $r \ll H$ ); only these small adapter weights are trained.
- *LoRA (Low-Rank Adaptation)*: freeze an existing weight matrix  $W$  entirely, and instead learn a separate low-rank correction  $\Delta W = BA$ , where  $A \in \mathbb{R}^{r \times d_{\text{in}}}$ ,  $B \in \mathbb{R}^{d_{\text{out}} \times r}$ , and  $r \ll \min(d_{\text{in}}, d_{\text{out}})$ . The effective weight used at inference is  $W' = W + BA$ ; only  $A$  and  $B$  are ever updated.

**Numerical example (why low rank saves so many parameters).** Suppose  $W \in \mathbb{R}^{4096 \times 4096}$  (a plausible size for one attention projection matrix in a mid-size LLM). Fully fine-tuning  $W$  directly means  $4096 \times 4096 \approx 16.78\text{M}$  trainable parameters for just that one matrix. LoRA with rank  $r = 8$ :  $A \in \mathbb{R}^{8 \times 4096}$  (32,768 parameters) plus  $B \in \mathbb{R}^{4096 \times 8}$  (32,768 parameters) = 65,536 trainable parameters total – about  $16.78\text{M}/65,536 \approx 256\times$  fewer trainable parameters than fully fine-tuning that one matrix, while still being able to express any rank- $\leq 8$  correction to it.

## 2.27 Pruning (p. 29)

**Core idea.** After training, remove some fraction of the network’s weights, then fine-tune to recover any accuracy lost.

**Two flavors.** *Magnitude pruning* (unstructured): remove individual weights with the smallest absolute value, producing a *sparse* weight matrix – this shrinks storage, but does not necessarily speed up computation on ordinary hardware, since GPUs are optimized for dense matrix multiplication, not arbitrary scattered sparsity patterns. *Structured pruning*: remove whole neurons, attention heads, or entire layers, keeping the remaining matrices fully *dense* (just smaller) – this does translate into real speedups on ordinary hardware, since a smaller dense matmul is simply faster.

**Recovery.** The slide notes that PEFT alone might not restore enough capacity after pruning (since pruning actually removes information, not just adds task-specific parameters), sometimes requiring a full, expensive fine-tune instead.

## 2.28 Knowledge Distillation (p. 30)

**Core idea.** Train a small “student” model to imitate a larger, already-trained “teacher” model’s full output distribution (“soft labels”), rather than training the student only against single correct hard labels.

**Mechanism.** The teacher generates a probability distribution over the vocabulary for each training example; the student is trained to match that distribution by minimizing the KL divergence between the two:

$$D_{\text{KL}}(P \parallel Q) = \sum_v P(v) \log \frac{P(v)}{Q(v)},$$

where  $P$  is the teacher’s distribution and  $Q$  is the student’s. This is standard background (Hinton et al., not one of the three papers provided for this session): matching the teacher’s *full* distribution – including which wrong answers it considered plausible – conveys more information per example than a single hard label would.

**Numerical example.** Toy 3-token vocabulary, teacher distribution  $P = [0.7, 0.2, 0.1]$ . If a student matches the teacher exactly,  $Q_a = [0.7, 0.2, 0.1]$ , then  $D_{\text{KL}}(P \parallel Q_a) = 0$  (identical distributions, zero divergence – a sanity check). If a weaker student is nearly uniform,  $Q_b = [0.34, 0.33, 0.33]$ :

$$D_{\text{KL}}(P \parallel Q_b) = 0.7 \ln \frac{0.7}{0.34} + 0.2 \ln \frac{0.2}{0.33} + 0.1 \ln \frac{0.1}{0.33} \approx 0.505 - 0.100 - 0.119 = 0.286 \text{ nats},$$

correctly showing a clearly positive loss for the worse student and zero loss for the perfect one.

**Examples from the slide.** DistilBERT: 40% smaller than BERT, retaining 97% of its accuracy. TinyLlama (1.1B parameters): trained by distilling from LLaMA 2’s outputs. Best suited, per the slide, to producing tiny, fast, specialized models (e.g. on-device chat).

## 2.29 Advantages of PEFT (p. 31)

The slide lists: increased efficiency, faster time-to-value, no catastrophic forgetting, lower risk of overfitting, lower data demands, more accessible AI, more flexible AI (attributed to IBM). Two of these follow directly from the mechanisms already covered above: *no catastrophic forgetting*, because the original pretrained weights stay completely frozen and cannot be overwritten by task-specific updates (a real risk with full fine-tuning, where large gradient updates can shift the whole network away from what it originally learned); and *lower risk of overfitting / lower data demands*, because only a small number of new parameters are being learned (recall the LoRA example above: 65,536 trainable parameters versus 16.78M for full fine-tuning of one matrix), leaving much less capacity to simply memorize a small task-specific dataset.

## 3 Key Formulas

- Cross-entropy loss (MLM and causal LM alike):  $L = -\log P(\text{correct token} \mid \text{context})$ .
- Scaled dot-product attention:  $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$ .
- Causal (autoregressive) factorization:  $p(w_1, \dots, w_T) = \prod_{j=1}^T p(w_j \mid w_1, \dots, w_{j-1})$ .

- Compute-optimal scaling (Chinchilla):  $C \approx 6ND$ ; compute-optimal  $N_{\text{opt}}$  and  $D_{\text{opt}}$  should scale in equal proportion as  $C$  grows.
- Relative positional attention (Shaw et al.):

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}, \quad z_i = \sum_j \alpha_{ij}(x_j W^V + a_{ij}^V).$$

- Rotary positional encoding (RoPE):  $f_{\{q,k\}}(x_m, m) = R_{\Theta, m}^d W_{\{q,k\}} x_m$ ; the key property,  $q_m \cdot k_n$  depends only on  $(m - n)$ .
- RMSNorm:  $\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_k x_k^2 + \epsilon}} \odot g$ .
- int8 affine quantization:  $x = S(x_q - Z)$ ,  $x_q = \text{round}(x/S + Z)$ .
- KL-divergence distillation loss:  $D_{\text{KL}}(P\|Q) = \sum_v P(v) \log \frac{P(v)}{Q(v)}$ .